# Introduction to Object-oriented Programming (OOP)

HDDS1103

Lecture 02

Data Types, Variables, basic operations

# Intended Learning Outcomes

- After completing the lesson, students should be able to:
  - Create variables and constants of different data types
  - Apply type conversion between different data types
  - Perform console-based Input/Output

# Variable and Constants

# Variable, Literal, and Constant

- Variable
  - named storage location in RAM
- Literal
  - a value of certain type.
- Constant
  - use `static` and `final` modifier
  - usually in UPPER_CASE.
  - separated by underscore (_)

# Example: VariableLiteralConstant.java

```
class VariableLiteralConstant{
    static final String PREFIX = "My age is: ";
    public static void main(String[] args) {
        int myAge = 18;
        System.out.print(PREFIX);
        System.out.println(myAge);
        System.out.print("A value of string");

    }
}
```

# Identifiers in Java

- Programmer defined names for variable, class, methods
- Identifier Rules
  - Can contain letters (A-Z,a-z), digits (0-9), underscores (_), and dollar signs ($)
  - Must start with a letter
    - Should start with lowercase letter (Uppercase for class name)
    - Can begin with _ and $ (not recommended)
  - Cannot contain white space
  - Case sensitive
  - Cannot use reserved words

Reference: https://www.w3schools.com/java/java_variables_identifiers.asp

# Reserved Words

| | | | | | |
|---|---|---|---|---|---|
| abstract | const | final | int | public | throw |
| assert | continue | finally | interface | return | throws |
| boolean | default | float | long | short | transient |
| break | do | for | native | static | true |
| byte | double | goto | new | strictfp | try |
| case | else | if | null | super | void |
| catch | enum | implements | package | switch | volatile |
| char | extends | import | private | synchronized | while |
| class | false | instanceof | protected | this | |
| _ (underscore) | | | | | |

true, false, and null are technically literals.

# Naming Convention

- Except for variables, all instance, class, and class constants are in mixed case with a lowercase first letter.
- Internal words start with capital letters.
- Variable names should be short yet meaningful.
- The choice of a variable name should be mnemonic- that is, designed to indicate to the casual observer the intent of its use.
- Common names for temporary variables (for temporary "throwaway")
  - Integer: i, j, k, m, and n
  - Characters: c, d, and e

More: https://www.oracle.com/java/technologies/javase/codeconventions-namingconventions.html

# Variable declaration

End with semi-colon

- Java is statically-typed
  - all variables must first be declared before they can be used.
- Syntax:

```
DataType variableName [= initial value];
```

  - stating the variable's type and name with optional initial value
- Example:

```
int myAge = 18;
String name;
```

# Declare and initialize variables

## Python (Dynamic typing)

```python
# Declare variable in python
number = 3
real_num = 3.14
course_code = "HDDS1103"
# can assign another data type
course_code = 1103
```

## Java (Static typing)

```java
// Declare variable in java
int number = 3;
double real_num = 3.14;
String course_code = "HDDS1103";
// can NOT assign another data type
course_code = 1103;
```

# Exercise - PrintYourName.java

- Print your name using variable
- Steps
  - Create the general Java file structure
  - Declare a variable (myName) of String type
  - Assign your name into the variable
  - Print the content of the name variable.

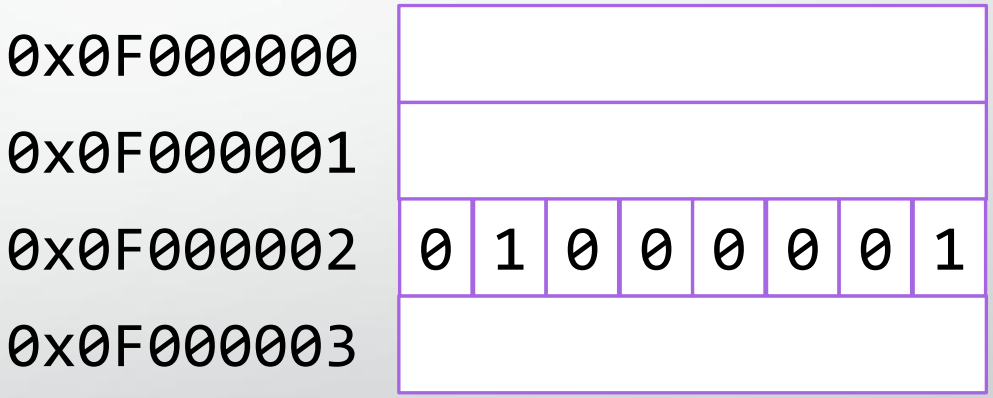
*Sample output*

```
My name is: Ben Fung.
```

# Data Types

# Bit and Byte

- Bit: Smallest unit
  - Store 0 or 1
- Byte: Collection of 8 bits
  - Store 0 to 255 (unsigned)
  - Can represent character
    - 'A' : $0100\ 0001_2 = 065_{10}$ (ASCII)



# Primitive data type

- Predefined data types by Java
  - Named by reserved words
- 8 primitive data types
  - Integer: `byte < short < int < long`
  - Floating-point number: `float < double`
  - Boolean (True / False) : `boolean`
  - Character (Unicode): `char`

https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html

# Character type (char)

- store a single 16-bit Unicode character.
  - From '\u0000' (or 0) to '\uffff' (or 65,535).
  - Size: 2 bytes (16 bits)
  - Default Value: '\u0000'.
- literals: denoted by single quotes
  - e.g. 'a', 'Z', ‘5’
  - Can use Unicode values like '\u0041' for 'A'.
- supports special escape sequences
  - '\n' for newline,
  - '\t' for tab,
  - '\\' for a backslash,
  - '\'' for a single quote,
  - '\"' for a double quote.

# Character type - Example

## Code

```
char ch1 = 'A';
char ch2 = '\u004A';
// Unicode for 'J'
// 0x4A (base 16) = 74 (base 10)
System.out.println("ch1: " + ch1);
System.out.println("ch2: " + ch2);
```

## Output

```
ch1: A
ch2: J
```

# Numeric Data Type

| Type | Size (byte ) | Range | Default |
|---|---|---|---|
| `byte` | 1 | Integers in the range of -128 to +127 | 0 |
| `short` | 2 | Integers in the range of -32,768 to +32,767 | 0 |
| `int` | 4 | Integers in the range of -2,147,483,648 to +2,147,483,647 | 0 |
| `long` | 8 | Integers in the range of -9,223,372,036,854,775,808 to +9,223,372,036,854,775,807 | 0L |
| `float` | 4 | Floating-point numbers in the range of $\pm 3.4 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$ with 7 digits of precision | 0.0f |
| `double` | 8 | Floating-point numbers in the range of $\pm 1.7 \times 10^{-308}$ to $\pm 1.7 \times 10^{308}$ with 15 digits of precision | 0.0d |

# Numeric Literals

- assign a value using (=) (i.e. No new keyword is needed)
- Special attention
  - For long - end with L or l
    - default to integer (int)
  - For float - end with F or f
    - Otherwise, default to double
- Can use underscore (_) for readability
- Scientific notation using E

More: https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html

# Example: NumericLiteral.java

```
byte b = 12;
short s = b;
int i = s;
long l = 1_234_000L + i;

float f = 1_234.56f;
double earth_mass = 5.97E24; // Earth mass: 5.97×10²⁴ kg
```

# Date Type Conversion

- When one primitive data type is assigned to another data type
- Type casting
  - Widening (Automatic): smaller to larger
  - Narrowing (Manual): larger to smaller,
    - possible loss precision
- Syntax:

```
targetVariable = (targetType) sourceVar;
```

https://www.w3schools.com/java/java_type_casting.asp

# Example: TypeCasting.java

```
long l = 1_234_000L ;
byte b = (byte) l;


double e = 2.718281828459045;
float f;
f = (float) e;
```

# Exercise - `TypeCastingExercise.java`

- Complete the provided Java template by using type casting

*Sample output*

```
100 (int) -> 100.0 (double)
9.78 (double) -> 9 (int)
A (char) -> 65 (int)
66 (int) -> B (char)
```

# Operators

NO **
use Math.exp()

# Arithmetic operators in Java

| Operation | Operator | Example | Result |
|---|---|---|---|
| Add | + | 3 + 4 | 7 |
| Subtraction | - | 2.5 – 9 | -6.5 |
| Multiplication | * | 3 * 4 | 12 |
| Integral Division | / | 7 / 3 | 2 |
| Floating-point division | / | 7 / 3.0 | 2.3333333333333335 |
| Exponential | Sorry, No operator | | |
| Modulus | % | 7 % 3 | 1 |

# Precedence & Associativity

- from highest precedence (most binding)
- to lowest precedence (least binding).
- Operators in the same box have the same precedence.
- Operators in the same box group left to right

# Arithmetic Operators

| Operators | Meaning |
|---|---|
| `()` | Parentheses |
| `+x, -x, ~x` | Unary plus, Unary minus, Bitwise NOT |
| `*, /, %` | Multiplication, Division, Modulus |
| `+, -` | Addition, Subtraction |

# Example: `ArithmeticExample.java`

| Examples | Result: |
| --- | --- |
| `2 + 3 * 4` | |
| `(2 + 3) * 4` | |
| `10 - 7 / 2` | |
| `(10 - -7) / 2` | |
| `5 + 2 % 3 - 1` | |
| `(5 + 2) % (3 - 1)` | |

# Bitwise Operators

| Operators | Meaning |
|---|---|
| <<, >> | Bitwise shift operators |
| & | Bitwise AND |
| ^ | Bitwise XOR |
| \| | Bitwise OR |

# Comparison and Logical Operators

| Operators | Meaning |
|---|---|
| ==, !=, >, >=, <, <= | Comparisons, Identity, Membership operators |
| ! | Logical NOT |
| && | Logical AND |
| \|\| | Logical OR |

# Example of Comparison & Logical Operators

| Comparison (e.g. a=10, b=20) |
| --- |
| a == b |
| a != b |
| a > b |
| a < b |
| a >= b |
| a <= b |

| Logical Operators (e.g. x= true, y= false) |
| --- |
| x && y |
| x \|\| y |
| !x |
| !y |

Sample Program in ComparisonAndLogicalOperators.java

# Assignment operators

- Shortcuts of assignment

| Operator | Example | Same as |
|---|---|---|
| = | x = 10 | x = 10 |
| += | x += 2 | x = x + 2 |
| -= | x -= 1 | x = x – 1 |
| *= | x *= 4 | x = x * 4 |
| /= | x /= 2 | x = x / 2 |
| %= | x %= 6 | x = x % 6 |
| ++ | x++<br>++x (prefix) | x = x + 1 |
| -- | x-- (postfix)<br>--x | x = x - 1 |

# Exercise: Calculating Body Mass Index (BMI)

- Calculate the BMI based on given weight and height.
- Instructions
  - Declare variables for weight (in kilograms) and height (in meters).
  - Calculate the BMI using the formula:
  - Print the calculated BMI.

$$BMI = \frac{weight}{height^2}$$

# Basic of programming - Output

# System class

- One of the Java API classes
- Contains useful classes
  - In: "Standard" input stream
  - Out: "Standard" output stream
  - Err: "Standard" error stream

# System.out class

- OutputStream class: A stream already open & ready to accept output data
- Can be display output / other destination
- `System.out.print`(data): print the data without newline
- `System.out.println`(data): print the data & a newline character (\n)

Details: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/System.html#out

# PrintStream.print() methods

| Modifier and Type | Method | Description |
|---|---|---|
| void | print(Object obj) | Prints an object |
| void | println(Object obj) | Prints an object and then terminates the line |
| PrintStream | printf(String format, Object... args) | Write a formatted string to output stream |

# Java Escape Sequences

| ES | Meaning | Description |
|---|---|---|
| \n | newline | Advances the cursor to the next line for subsequent printing |
| \t | tab | Causes the cursor to skip over to the next tab stop |
| \b | backspace | Causes the cursor to back up, or move left, one position |
| \r | carriage return | Causes the cursor to go to the beginning of the current line, not the next line |
| \\ | backslash | Causes a backslash to be printed |
| \’ | single quote | Causes a single quotation mark to be printed |
| \” | double quote | Causes a double quotation mark to be printed |

# Format String for `System.out.printf()`

- Like printf() in C/C++,
- we use format specifiers start with `%` sign
- Can specify
  - Width (e.g. `%10s`)
  - Precision of a floating point (e.g. `%.2f`)
  - Width & precision (e.g. `%5.2f`)

| Specifier | Date Type applicable |
| --- | --- |
| %s | String. Other type will convert into string before display. |
| %d | decimal (For integer) |
| %f | floating-point type |
| %c | character type |
| %b | boolean type. |
| %e | For scientific notation |
| %% | Output single percent sign |

# Format String - Example

```java
int i = 10;
float f = 20.5f;
char c = 'a';
String str = "Hello World";
System.out.printf("Integer : %4d\n", i);
System.out.printf("Float : %.2f\n", f);
System.out.printf("Character : %c\n", c);
System.out.printf("String : %s\n", str);
```

```
Integer :   10
Float : 20.50
Character : a
String : Hello World
```

# System.out.printf() example (PrintDreamGPA.java)

```
public class PrintDreamGPA {
    public static void main(String[] args) {
        System.out.print("My name is \t\t  :");
        System.out.println(" Ben.");
        System.out.printf("\t Last Semester GPA: %5.3f.\n", 2.71828);
        System.out.printf("%8s This Semester GPA: %5.3f.", "(Hope)", 3.14156);
    }
}
```

# String concatenation (+)

- When one of the operand is a text (String),
  - plus (+) operator will 'glue' the results as string
- Example:

```
String program = "HDDS";
int code = 1103;
System.out.println("This is " + program + code + ".");
```

More: https://www.w3schools.com/java/java_strings_concat.asp

# Basic of programming - Input

# System.in class

- InputStream class
  - Already open & ready to supply input data
- Corresponds to keyboard input / other input source
- Can be wrapped in a InputStreamReader
  - Read unicodes one-by-one

More: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/System.html#in

# `Scanner` Class

- To read input from the keyboard
- itis defined in `java.util`, we need to import before use it:

```
import java.util.Scanner;
```

- Scanner objects work with `System.in`
- To create a `Scanner` object:

```
Scanner kb = new Scanner(System.in);
```

# Scanner Class methods

| Method name | Explanation |
|---|---|
| nextLine() | Advances this scanner past the current line and returns the input as String. |
| nextInt() | Scan the next token of the input as an int (Integer) |
| nextDouble() | Scan the next token of the input as a double value |
| hasNextInt() | Return true if the next token can be interpreted as an int |
| hasNextDouble() | Return true if the next token can be interpreted as a double |
| close() | Closes the scanner. (Remember to close **at last**) |

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Scanner.html

# Example (ReadName.java)

```
import java.util.Scanner;  // Import the Scanner class
public class ReadName {
  public static void main(String[] args) {
    Scanner kb = new Scanner(System.in);  // Create a Scanner object
    System.out.print("Please enter your name: "); // Prompt
    String name = kb.nextLine();  // Read until newline

    System.out.println("Your name is: " + name);  // Report back
    kb.close();
  }
}
```

CLOSE the Scanner

# Close the Scanner Objects

## Resource Management

- Frees system resources (memory, file handles)
- Prevents memory leaks and resource exhaustion

## File and Network Handles

- Ensures files & network connections are released
- Avoids blocking access for other programs

## Flushing Buffers

- Ensures buffered input is properly flushed
- Performs necessary cleanup tasks

# Exercise – Meet a new friend

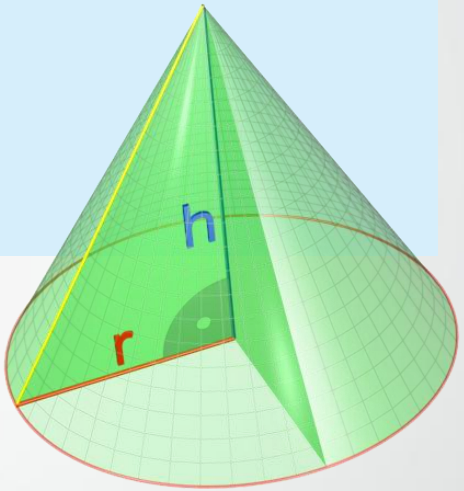
## Requirements

- Write a program to meet a friend
  - Ask for her name
  - Ask for her age
    - use String by now
    - (can use int if possible)
  - Print the name and age

## Sample output

```
May I know your name? Alice
Hi Alice, how old are you? 25
I just meet a new friend: Alice
The age is: 25
```

# Exercise - Volume of cone




$$V = \frac{1}{3}\pi r^2 h$$

- Calculate the volume of a cone.
- Steps
  - 1. Ask for the height (h)
  - 2. Ask for the radius (r)
  - 3. Calculate the volume (V)
  - 4. Output the result.
- P.S. Assume pi = 3.1415

*Sample output*

```
Please input the height: 3.0
Please input the radius: 4
Volume of a cone with h(3.0) & r(4.0) is 50.26.
```

```
Please input the height: 5
Please input the radius: 7.0
Volume of a cone with h(5.0) & r(7.0) is 256.56.
```